Automated E-Commerce Deployment Platform

Team Members:

Rahma Ahmed Elsayed

Ahmed Abdelmohsen Abdelraheem

Mohamed Mohsen Abdelfatah

Project Proposal

Overview

The project is a microservices-based e-commerce web application built using Node.js, React, MongoDB, and Docker. It consists of multiple services (User, Product, Cart) communicating via REST APIs. The system is containerized for easier deployment and scalability.

Objectives

- Build a scalable and modular e-commerce system
- Ensure high availability and performance using Docker
- Apply DevOps practices (CI/CD, containerization, monitoring)
- Prepare the project for cloud deployment

Scope

The project covers backend microservices, frontend UI, and containerized deployment. It includes authentication, product management, and cart functionality. Future scope may include order/payment services and cloud hosting.

---

2. Project Plan

Timeline

Deliverable	Realistic Time to Learn (Beginner)
Dockerizing multi-service app	**1–2 weeks**
CI/CD with Jenkins	**1–2 weeks**
Kubernetes + auto-scaling	**3–5 weeks**
Ansible server config	**1–2 weeks**
Prometheus + Grafana dashboards	**1 week**
Terraform (AWS EC2, RDS, S3, ELB)	**2–3 weeks**
Automated rollback strategies	**1–2 weeks**
Nginx reverse proxy	**3–5 days**

Milestones

- Application runs locally (Docker)
- All services communicate correctly
- New features implemented
- Deployment completed (cloud/local server)

Deliverables

- Fully working microservices app
- Dockerized environment (docker-compose)
- CI/CD pipeline (jenkins)
- Documentation (README + architecture)

---

3. Task Assignment & Roles

DevOps Engineer (Primary Role)

Responsible for Docker setup, container orchestration, CI/CD pipelines, and deployment. Also handles environment variables, logging, and monitoring tools.

Backend Developer

Manages microservices (User, Product, Cart), builds APIs, and ensures proper database integration. Handles authentication and business logic.

Frontend Developer

Works on React frontend, integrates APIs, and improves UI/UX. Ensures proper communication with backend services.

In your case: you can position yourself as **DevOps + Backend hybrid**.

---

4. Risk Assessment & Mitigation Plan

Risks

- Service communication failure (API errors)
- Misconfigured Docker containers

- Security issues (JWT, exposed APIs)
- Deployment failures or downtime

Mitigation

- Use API testing tools (Postman) for validation
- Validate Docker configs and use docker-compose
- Implement authentication middleware & env variables
- Use CI/CD pipelines to test before deployment

---

5. KPIs (Key Performance Indicators)

Performance Metrics

- API response time < 300ms
- Page load time < 2 seconds
- Efficient container startup time

Reliability Metrics

- System uptime ≥ 99%
- Error rate < 2%
- Successful deployment rate (CI/CD)

Usage Metrics

- Number of active users
- Cart operations success rate
- User login success rate

System Analysis & Design

1. Problem Statement

Traditional monolithic e-commerce platforms often suffer from "all-or-nothing" failures; if the payment module crashes, the entire store goes down. Furthermore, scaling a monolith requires scaling the entire codebase, even if only the "Product Search" feature is under heavy load. This results in inefficient resource usage and slower deployment cycles for new features.

a. **Objectives**

A scalable game store platform allowing users to browse, purchase, and manage digital game licenses.

b. **Use Case & Descriptions**

System Actors

- **Guest:** Can browse games, view details, and search the catalog.
- **Registered User:** Can manage a profile, add games to a cart, and complete purchases.
- **Admin:** Can manage game inventory (CRUD), view sales reports, and manage user accounts.

Core Interactions

- **UC-01 (User Authentication):** User logs in via the Auth Service to receive a JWT.
- **UC-02 (Manage Inventory):** Admin adds a new "Game Title" to the Product Service.
- **UC-03 (Process Order):** User sends cart data to the Order Service, which validates the stock with the Product Service before confirming.

c. **Functional & Non-Functional Requirements**

Functional Requirements

- **FR-1 (Auth):** The system must provide secure JWT-based authentication and role-based access control (RBAC).
- **FR-2 (Catalog):** The system must allow users to filter games by genre, price, and platform.
- **FR-3 (Cart):** The system must persist cart data across sessions for registered users.

Non-Functional Requirements

- **Scalability:** The system must handle a 50% increase in traffic by spinning up additional Docker containers for the API Gateway.
- **Performance:** Catalog queries should return results in under 200ms.
- **Security:** All sensitive data (passwords) must be hashed using bcrypt before storage in MongoDB.

d. Software Architecture

Architecture Style: Microservices

The project follows a **decentralized architecture** where each business capability is a standalone service.

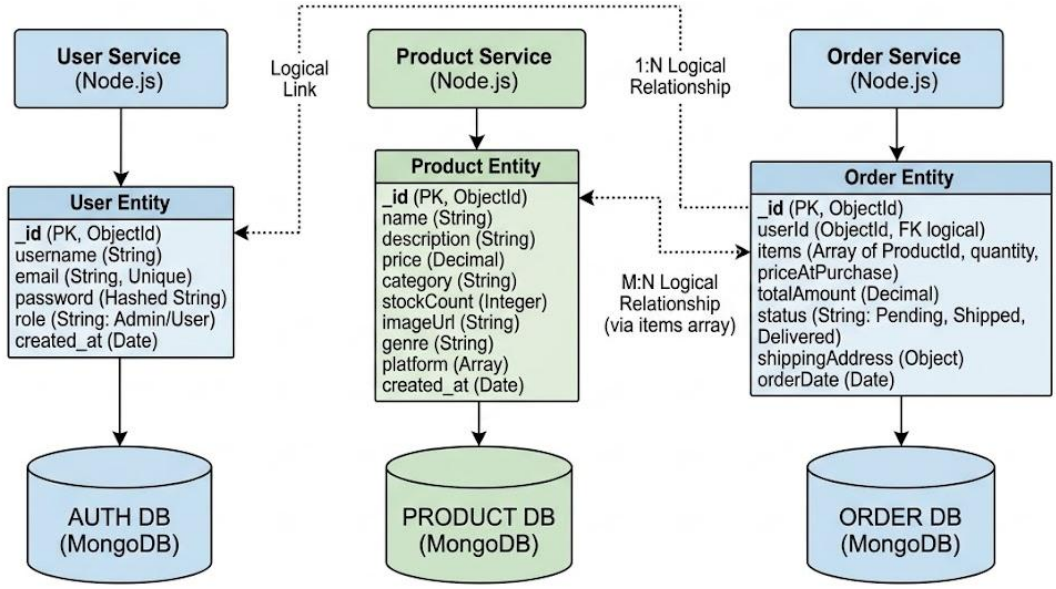
High-Level Components

1. **Presentation Layer (React/Vite):** A single-page application (SPA) that handles the UI and client-side routing.
2. **API Gateway (Nginx or Express-Gateway):** Acts as a reverse proxy. It intercepts all incoming requests from the frontend and routes them to the appropriate backend service based on the URL path (e.g., /api/auth/ -> Auth Service).
3. **Service Layer (Node.js/Express):**
    - **Auth Service:** Isolated logic for users and sessions.
    - **Product Service:** Manages game metadata.
    - **Order Service:** Manages transaction history.
4. **Data Layer (Per-Service Database):** To ensure loose coupling, each service has its own **MongoDB instance/collection**. No service directly queries another service's database.

---

2. Database Design & Data Modeling

The project uses **MongoDB** (NoSQL), but follows a relational data modeling approach within each service to maintain integrity.

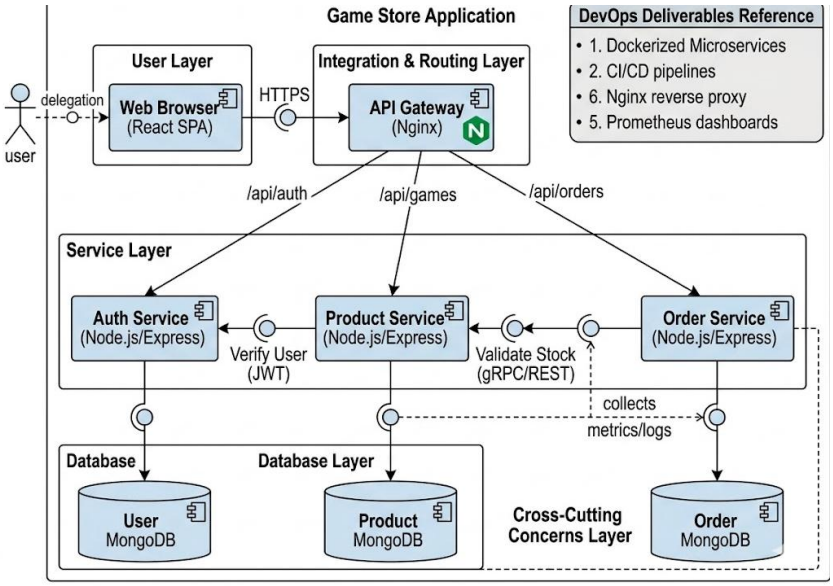


---

5. **System Deployment & Integration**
   - **Technology Stack**

Layer	**Technology**	**Purpose**
Frontend	React.js (Vite)	Client-side UI and state management.
Backend	Node.js / Express	Modular microservices for business logic.
Database	MongoDB	NoSQL data storage (separate instance per service).
API Gateway	Nginx	Reverse proxy, SSL termination, and routing.
Orchestration	Kubernetes	Managing, scaling, and healing containers.
Infrastructure	Terraform / Ansible	Infrastructure as Code (IaC) and configuration.
Monitoring	Prometheus / Grafana	Real-time metrics and visualization.

   - **Component Diagram**



---

6. Additional Deliverables

- **API Documentation**

  - POST /api/auth/register - Create a new user account.
  - GET /api/products - Fetch all available games.
  - POST /api/orders - Create a new purchase (requires Auth header).

- **Deployment Strategy**

  1. Hosting Environment

  The platform is hosted on **Amazon Web Services (AWS)** to leverage its global infrastructure and managed services.

  - **Compute: Amazon EC2** manages the lifecycle of the MERN containers.
  - **Networking:** A custom **VPC** with Public subnets (for the Nginx Load Balancer) and Private subnets (for the microservices and MongoDB) to ensure a "zero-trust" security model.
  - **Storage: Amazon S3** stores static game assets (images/videos), while **Amazon EBS** provides persistent storage for database volumes.
  - **Database: RDS** within the private subnet.

  2. Deployment Pipelines (CI/CD)

  The pipeline automates the journey from a code push in repo to a live production update.

Pipeline Stages (Jenkins)

1. **Source:** Detects changes in the main branch.
2. **Build & Test:** build and test for each microservice.
    - Compiles the React frontend.
3. **Security Scan:** Uses **Trivy** to check Docker images for vulnerabilities.
4. **Artifact Storage:** Builds Docker images and pushes them to **Docker Hub.**
5. **Provision: Terraform** checks if any infrastructure changes (like new subnets or load balancers) are required.
6. **Deploy:** Updates the Kubernetes manifests to pull the latest images.

3. Scaling Considerations

To handle the variable traffic patterns of an e-commerce store (e.g., flash sales or new game releases), the system implements Horizontal Pod Autoscaler (HPA)**:** Automatically increases or decreases the number of "Pods" (containers) for a specific service based on CPU or Memory usage. **Threshold:** If the Product service exceeds **70% CPU utilization**, HPA triggers a new pod within 60 seconds.

4. Reliability & Rollback Strategy

- **Rolling Updates:** Kubernetes replaces old pods with new ones one-by-one to ensure zero downtime.
- **Health Probes:**
    - **Liveness Probes:** Restarts a container if it crashes or hangs.
    - **Readiness Probes:** Ensures the service is fully started before the Load Balancer sends it any user traffic.
- **Automated Rollback:** If the new version fails its readiness probe, the pipeline automatically halts and reverts the Kubernetes deployment to the previous stable image tag.